

בקרת זרימה [= flow control]

בקרת זרימה (Flow Control) היא אחד משני הנדבכים המרכזיים בניהול הקשר של TCP (לצד בקרת עומסים - Congestion Control). בעוד שבקרת עומסים מגינה על הרשת מפני הצפה, בקרת זרימה מגינה על הנמען (השרת או הלקוח) מפני הצפה במידע שהאפליקציה שלו אינה מסוגלת לעבד. להלן ההסבר המלא והמקיף.

1. בקרת זרימה (Flow Control) – ההגדרה הפדגוגית

ההגדרה: בקרת זרימה היא מנגנון המבטיח שהשולח (Sender) לא ישלח נתונים בקצב העולה על יכולת הקליטה והעיבוד של הנמען (Receiver).

למה זה קריטי?

בארכיטקטורת המחשב המודרנית, הקרנל מעביר נתונים מהרשת אל חוצץ הקבלה (sk_rcvbuf). אם האפליקציה (כמו שרת Apache) "עסוקה" ולא קוראת את הנתונים מהחוצץ בקצב מספק, החוצץ יתמלא. ברגע שהחוצץ מלא, כל סגמנט חדש שיגיע מהרשת – ייזרק לפח (Packet Drop). TCP מונע זאת באמצעות "עצירה" או "האטה" מבוקרת של השולח.

2. הפרמטר rwnd (Receive Window)

ה-rwnd הוא ה"מד" שבאמצעותו הנמען משדר לשולח: "יש לי עוד X בתים פנויים בחוצץ הקבלה".

מהו ערכו ההתחלתי?

- ערכו נקבע במהלך שלב הקמת החיבור (3-Way Handshake):
- ב-SYN-ACK ששולח השרת, וב-SYN ששולח הלקוח, שניהם מפרסמים את גודל חוצץ הקבלה המקסימלי שהם הקצו (sk_rcvbuf).
 - זהו הגודל המקסימלי שהם מוכנים לאחסן בזיכרון הליבה עבור החיבור הזה.

מתי ואיך הוא משתנה?

ה-rwnd אינו ערך סטטי. הוא משתנה בכל סגמנט אישור (ACK) שנשלח מהנמען לשולח, בהתבסס על הנוסחה:

רשתות תקשורת, תשפ"ו
Flow Control
עמוד 1

$$rwnd = \text{ReceiveBufferCapacity} - (\text{LastByteReceived} - \$\$LastByteReadByApplication)$$

- **הגדלת ה-rwnd:** מתרחשת כאשר האפליקציה (Apache/Chrome) מבצעת (recv) ומרוקנת את החוצץ. הקרנל "מפנה מקום" ומעדכן את ה-rwnd ב-ACK הבא.
- **הקטנת ה-rwnd:** מתרחשת כאשר הנתונים מגיעים מהרשת בקצב מהיר יותר מכפי שהאפליקציה קוראת אותם. ככל שהחוצץ מתמלא, ה-rwnd הולך וקטן.

3. דוגמאות לשינוי הערך

נניח חוצץ קבלה של 10,000 בתים:

1. **מצב א' (חוצץ פנוי):** אפליקציה קראה את כל המידע. $rwnd = 10,000$. השולח רשאי לשלוח עד 10,000 בתים.
2. **מצב ב' (הגעת נתונים):** הגיעו 4,000 בתים. האפליקציה טרם קראה אותם. החוצץ תפוס ב-4,000. ה-ACK הבא ידווח: $rwnd = 6,000$.
3. **מצב ג' (האטה באפליקציה):** האפליקציה איטית. הגיעו עוד 6,000 בתים. ה-ACK הבא ידווח: $rwnd = 0$.

מה קורה כש $rwnd = 0$?

זהו מצב קריטי הנקרא **Zero Window**. השולח עוצר לחלוטין את השידור. הוא אינו רשאי לשלוח יותר נתונים, שכן ה-Receiver הצהיר שאין לו מקום פנוי בזיכרון. השולח נכנס למצב של "המתנה פעילה".

4. מקרי קצה והתמודדות

מקרה ה-Zero Window Probe: אם ה-Receiver שלח $rwnd=0$ ועצר את התעבורה, איך הוא יודיע ל-Sender מתי אפשר לחזור לעבוד?

- אם ה-Receiver יקרא נתונים מהחוצץ, הוא ישלח חבילת "עדכון חלון" (Window Update).
- אבל מה אם חבילת העדכון הזו הולכת לאיבוד ברשת? השולח לעולם לא ידע שהחוצץ התפנה!
- **הפתרון:** השולח משדר סגמנט זעיר (1 בית של נתונים) הנקרא **Zero Window Probe**. ה-Receiver חייב להשיב ל-Probe הזה עם ה-rwnd העדכני שלו, וכך החלון נפתח מחדש.

- **מצב של "Silly Window Syndrome":** מה קורה אם ה-Receiver מפרסם $rwnd$ של 1 בתים בלבד בכל פעם? השולח ישלח סגמנט של 1 בייט עם כותרת של 40 בתים (TCP+IP). זו בזבזנות נוראית.
- **הפתרון:** אלגוריתם של Clark – ה-Receiver לא יפרסם חלון חדש עד שלא יתפנה בחוצץ מקום לפחות בגודל MSS או חצי מגודל החוצץ הכולל.

Window Scaling

כדי להבין את ה-Window Scaling ואת הקשר ההדוק שבין ה-rwnd למנגנוני התזמון ב-Linux 6.14, עלינו להבין שהמערכת אינה רק "שומרת נתונים", אלא מבצעת אופטימיזציה של ניצול רוחב פס (Bandwidth-Delay Product).

1. Window Scaling (RFC 7323) – הרחבת גבולות ה-rwnd

ב-TCP הקלאסי, שדה ה-Window Size בכותרת הוא בן 16-ביט בלבד. המשמעות: $2^{16} = 65,535$ בתים.

הבעיה: ברשתות מודרניות של 10 Gbps ומעלה, החלון הזה מתמלא בתוך מילי-שניות בודדות, והשולח "נחנק" וממתין ל-ACK, למרות שהרשת יכולה להעביר הרבה יותר.

- **הפתרון: Window Scaling Option.** במהלך ה-3-way handshake שני הצדדים מחליפים ערך בשם Shift Count (ערך בין 0 ל-14).
- **המשמעות:** החלון האמיתי מחושב כך:

$$WindowSize_{actual} = WindowSize_{header} \times 2^{ShiftCount}$$

- **דוגמה:** אם השולח והנמען הסכימו על Shift Count של 7, אז ערך ה-65,535 הופך ל-8,388,480 בתים (8MB). זה מאפשר למערכות תקשורת להעביר כמויות עצומות של נתונים לפני שהן חייבות לקבל אישור (ACK).

2. הקשרים שבין rwnd למנגנוני התזמון (Linux 6.14)

בלינוקס מודרני, ה-rwnd אינו פסיבי. הוא "מדבר" עם ה-Scheduler של הקרנל ועם ה-Interrupts.

א. תלות ב-SoftIRQ ו-NAPI

כאשר מגיעים סגמנטים שממלאים את החוצץ, הקרנל מפעיל את ה-SoftIRQ (Interrupt Request).

- אם ה-rwnd קטן מאוד, הליבה "תופסת את החרדה" שהיא עומדת להגיע ל-Zero Window. לכן, היא מתזמנת את תהליכון ה-ksoftirqd בעדיפות גבוהה יותר כדי לפנות את ה-sk_rcvbuf אל ה-User Space (האפליקציה).
- **הקשר:** ה-rwnd הוא ה"מד" שקובע כמה אגרסיבי הקרנל יהיה בשימוש במשאבי CPU כדי לעבד את ה-Interrupts של כרטיס הרשת.

ב. Delayed ACK Timer ו-rwnd

כפי שראינו בתסריט א', ה-Delayed ACK מעכב את האישור (נניח ל-40ms) כדי לחסוך בתעבורה.

- **התנגשות:** אם ה-rwnd מתחיל להתקרב לאפס, הקרנל ב-Linux 6.14 לא ימתין לטיימר. הוא מבטל את ה-Delayed ACK באופן מיידי ושולח ("Window Update") ACK כדי למנוע מצב שהשולח יחסום את עצמו בגלל חוסר מידע. זהו מנגנון תעדוף: **זמינות הזיכרון (Flow Control) גוברת על היעילות האנרגטית (Delayed ACK).**

ג. הרצף המתוזמן: התעורות ה-recv()

- כאשר האפליקציה מבצעת recv(), היא לא רק מעתיקה נתונים, היא מפעילה **Wait Queue**.
- אם ה-rwnd היה אפס (Zero Window), האפליקציה המקבלת הודיעה ל-TCP שהיא "עסוקה".
- ברגע שהאפליקציה קוראת את הנתונים, ה-sk_rcvbuf מתרוקן, וה-Kernel מתזמן את ה-TCP להוציא **Window Update**.
- **הדיוק ב-Linux 6.14:** הקרנל משתמש ב-tcp_data_queue כדי לוודא שברגע שיש "מרווח נשימה" ב-rwnd, הוא מדווח על כך מיד לשולח, כדי שהצינור (Pipeline) של הנתונים לא יישאר יבש (Idle).

3. מקרה קצה: ה-rwnd וה-Congestion Control (משחק כוחות)

- השולח תמיד בוחר את ה-Effective Window לפי:

$$\text{Effective Window} = \min(\text{rwnd}, \text{cwnd})$$
 ב-Linux 6.14, קיים מנגנון אדפטיבי:
- אם ה-rwnd קטן (הנמען איטי), הקרנל של השולח "מרדים" את ה-cwnd (בקרת עומסים) כי אין טעם לנסות להציף רשת אם הנמען ממילא לא מסוגל לקלוט.
- הקרנל מתזמן את ה-cwnd להיבנות מחדש (Slow Start) **בדיוק לפי קצב הגדילה של ה-rwnd**.

סיכום:

- ה-rwnd הוא ה"בלם" של המערכת. הוא מנוהל על ידי הקרנל באמצעות דפי זיכרון וירטואליים, ומתואם עם טיימרים (כמו ה-Delayed ACK) כדי להבטיח זרימה רציפה. ב-Linux 6.14, המערכת כל כך מתוחכמת שהיא יודעת לוותר על אופטימיזציות (כמו ביטול Delayed ACK) ברגע שהיא מזהה סכנה שהזיכרון יתמלא.